

Self-Attention Mechanism

What problem does it solve?

Traditional RNNs process words one at a time, making it difficult to capture relationships between distant words.

Example:

"The animal didn't cross the street because it was tired."

The model needs to understand that **"it"** refers to **"animal"**, not **"street."**

Self-attention allows every word to **look at every other word** in the sentence simultaneously.

Example

Sentence:

The cat sat on the mat

When processing the word **sat**, the model asks:

- Which words are important?
- Should I pay more attention to **cat**?
- Is **the** important?
- Is mat relevant?

It computes an **attention score** for every word.

Example attention weights:

Word

Attention Weight

The

0.05

Cat

0.55

Sat

0.20

Word

Attention Weight

On

0.08

The

0.02

Mat

0.10

The output representation of **sat** becomes a weighted combination of all the word representations, with **cat** contributing the most.

Three vectors

Every token is converted into three vectors:

- **Query (Q)** – What information am I looking for?
- **Key (K)** – What information do I contain?
- **Value (V)** – The information I provide.

For each token:

1. Compare Query with every Key.
2. Compute similarity scores.
3. Apply Softmax to convert scores into probabilities.
4. Use these probabilities to weight the Value vectors.
5. Sum the weighted Values to produce the new token representation.

This allows the model to capture context dynamically.

Vector Embedding

What is an embedding?

Computers cannot process words directly.

Instead, each word is represented as a list of numbers called an **embedding vector**.

Example:

Dog

↓

[0.21, -0.87, 0.45, 1.02, ...]

The embedding captures semantic meaning.

For example:

Word

Similar To

King

Queen

Dog

Puppy

Paris

France

Words with similar meanings have embeddings that are close together in the vector space.

Why embeddings matter

Instead of treating words as IDs:

Dog = 1

Cat = 2

Car = 3

Embeddings capture relationships:

Dog → [0.4, 0.7, ...]

Cat → [0.5, 0.6, ...]

Car → [-0.8, 0.1, ...]

Here, **Dog** and **Cat** are closer to each other than **Car**.

Transformer pipeline

Text

↓

Tokenizer

↓

Token IDs

↓

Embedding Layer

↓

Dense vectors

↓

Transformer Layers

Feed-Forward Neural Network (FFN)

After self-attention updates each token's representation, the Transformer passes each token independently through a small neural network called the **Feed-Forward Network**.

A typical FFN consists of:

Linear Layer



Activation Function (e.g., GELU or ReLU)



Linear Layer

Example:

Input vector



Expand dimensions



Nonlinear activation



Reduce back to original size

Why is it needed?

Self-attention mixes information between tokens.

The FFN then performs nonlinear transformations on each token's representation, helping the model learn more complex patterns and relationships.

Analogy

Imagine reading a group project report.

- **Self-attention** is like discussing with every teammate to understand the overall context.
- **Feed-Forward Network** is like each teammate refining and improving their own section after the discussion.

Together, they produce a better final result.

How these three components work together

Sentence



Tokenizer

|



Embeddings

|



Self-Attention

|



Feed-Forward Network

|



Repeat (many Transformer layers)

|



Output

Key takeaways

- **Self-attention** lets every token attend to every other token, enabling the model to capture context and long-range dependencies.
- **Vector embeddings** convert words or tokens into dense numerical vectors that encode semantic relationships.
- **Feed-Forward Neural Networks** apply nonlinear transformations to each token after attention, enriching its representation before passing it to the next Transformer layer.

Studying these three concepts in this order—**embeddings → self-attention → feed-forward network**—provides a strong foundation for understanding modern Transformer-based models such as GPT, Claude, Gemini, and Llama.